

Mini ERP + CRM Operations Portal

Project Documentation

A full-stack internal operations system for wholesale/distribution companies

Built with NestJS, React, PostgreSQL, Prisma, and Tailwind CSS

Date: 29 July 2026

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Database Schema
5. Features & Modules
6. API Endpoints Reference
7. Roles & Permissions Matrix
8. Setup Instructions
9. Deployment Guide
10. Test Credentials
11. Assumptions & Design Decisions
12. Known Limitations
13. Submission Checklist

1. Project Overview

The Mini ERP + CRM Operations Portal is a full-stack internal operations system built for a wholesale/distribution company. It combines Enterprise Resource Planning (ERP) and Customer Relationship Management (CRM) functionality into a single web application.

The system enables sales, warehouse, and accounts teams to manage customer relationships, track product inventory, record stock movements, and create sales challans with automated stock deductions. It was developed as a case-study assignment for a FullStack Developer recruitment process.

Live URLs:

- Frontend: <https://erp-crm-operations-portal.vercel.app>
- Backend API: <https://erp-crm-api.onrender.com>
- Database: Neon (PostgreSQL 16, pooled connection)
- GitHub Repo: <https://github.com/raj-singh1802/erp-crm-operations-portal>

2. Technology Stack

Layer	Technology
Backend Framework	NestJS 11.x (TypeScript)
Database	PostgreSQL 16 with Prisma 5.x ORM
Frontend	React 19 + TypeScript, Vite 8
Styling	Tailwind CSS 4 + PostCSS
Authentication	JWT (Access + Refresh token pair)
PDF Generation	pdfkit
State Management	Zustand 5
HTTP Client	Axios 1.x
Form Handling	react-hook-form 7.x
Containerization	Docker + docker-compose
CI/CD	GitHub Actions
Hosting	Vercel (frontend), Render (backend)

3. System Architecture

The application follows a two-package monorepo structure with a NestJS backend API and a React + Vite frontend SPA. The backend adopts NestJS modular architecture with clear separation of concerns.

3.1 Backend Module Structure

src/	
auth/	JWT login/refresh/logout, Guards, Decorators
users/	User CRUD (internal, minimal)
customers/	Customer CRM + Follow-up notes
products/	Product CRUD + Stock movements (w/ transactions)
challans/	Sales challan draft/confirm/cancel + PDF export
prisma/	Shared PrismaService (injectable DB client)
common/	Shared Guards, Decorators, Exception Filters

3.2 Frontend Structure

src/	
api/client.ts	Axios with auto-refresh interceptor
components/	ProtectedRoute, ToastContainer
layouts/	DashboardLayout (sidebar + topbar)
pages/login/	Login page
pages/dashboard/	Dashboard with low-stock/draft widgets
pages/customers/	List, Form, Detail with follow-up notes
pages/products/	List, Form, Detail with stock movements
pages/challans/	Builder, List, Detail with PDF download
stores/	Zustand stores (authStore, toastStore)
types/index.ts	TypeScript interfaces

3.3 Data Flow

Client -> JwtAuthGuard -> RolesGuard -> Controller -> Service -> Prisma -> PostgreSQL

The frontend mirrors this: ProtectedRoute blocks unauthenticated access, then role-based UI gating hides action buttons per role.

3.4 Confirm Challan Transaction Design

- 1. Wrapped in a Prisma \$transaction with row-level locking (SELECT FOR UPDATE)
- 2. Items sorted by productId to prevent deadlocks between concurrent confirmations
- 3. Each product currentStock >= quantity verified; failure rolls back entire transaction
- 4. On success: stock decremented, StockMovement logged, product name & price snapshotted
- 5. No partial state - zero stock deducted on any failure path

4. Database Schema

The database uses PostgreSQL 16 with Prisma ORM. Seven core models and five enums define the data layer.

Entity	Key Fields
User	id, name, email (unique), passwordHash, role, refreshTokenHash, createdAt
Customer	id, name, mobile, email, businessName, gstNumber, customerType, address, status, followUpDate, createdAt, updatedAt
FollowUpNote	id, note, customerId (FK), createdBy (FK), createdAt
Product	id, name, sku (unique), category, unitPrice, currentStock, minStockAlert, warehouseLocation, imageUrl, createdAt, updatedAt
StockMovement	id, productId (FK), quantityChanged, movementType, reason, createdBy (FK), createdAt
Challan	id, challanNumber (unique), customerId (FK), status, totalQuantity, createdBy (FK), createdAt
ChallanItem	id, challanId (FK), productId (FK), quantity, productNameSnapshot, unitPriceSnapshot

Enums

- Role: ADMIN, SALES, WAREHOUSE, ACCOUNTS
- CustomerType: RETAIL, WHOLESALE, DISTRIBUTOR
- CustomerStatus: LEAD, ACTIVE, INACTIVE
- MovementType: IN, OUT
- ChallanStatus: DRAFT, CONFIRMED, CANCELLED

5. Features & Modules

Authentication

JWT access + refresh token pair with rotation. Refresh tokens stored as bcrypt hashes in the database. Auto-refresh on 401 via Axios interceptor and Passport JWT strategy.

Role-Based Access Control

4 roles - Admin, Sales, Warehouse, Accounts. Backend enforces via `@Roles()` decorator + RolesGuard. Frontend gates routes via ProtectedRoute component.

Customer CRM

Full CRUD with pagination, search by name/mobile, filter by status and type. Follow-up notes with timestamps and author tracking.

Product & Inventory

CRUD with unique SKU enforcement. Stock movement log with atomic IN/OUT adjustments via Prisma `$transaction`. Low-stock flag computed at query time.

Sales Challans

Draft -> Confirm -> Cancel workflow. Auto-generated challan number CH-YYYY-XXXXXX. Confirmation uses `$transaction` with row-level locks sorted by productId to prevent deadlocks. Insufficient stock returns 400 naming short products.

PDF Invoice Export

GET `/challans/:id/pdf` generates a PDF via pdfkit with company header, customer info, itemized table with line totals.

Product Image Upload

POST `/products/:id/image` accepts multipart uploads to local storage. S3-ready abstraction documented for production switch.

Dashboard

Widget showing low-stock products count and draft challan counts.

6. API Endpoints Reference

A complete Postman collection with example requests is available at [postman/ERP_CRM_Portal.postman_collection.json](#).

Auth

Method	Path	Auth	Roles
POST	/auth/login	No	All
POST	/auth/refresh	No	All
POST	/auth/logout	Yes	All

Customers

Method	Path	Auth	Roles
POST	/customers	Yes	Admin, Sales
GET	/customers	Yes	All
GET	/customers/:id	Yes	All
PATCH	/customers/:id	Yes	Admin, Sales
POST	/customers/:id/follow-ups	Yes	Admin, Sales

Products

Method	Path	Auth	Roles
POST	/products	Yes	Admin
GET	/products	Yes	All
GET	/products/:id	Yes	All
PATCH	/products/:id	Yes	Admin, Warehouse
POST	/products/:id/image	Yes	Admin, Warehouse
GET	/products/:id/stock-movements	Yes	All
POST	/products/:id/stock-movements	Yes	Admin, Warehouse

Challans

Method	Path	Auth	Roles
POST	/challans	Yes	Admin, Sales
GET	/challans	Yes	All
GET	/challans/:id	Yes	All
GET	/challans/:id/pdf	Yes	Admin, Sales
PATCH	/challans/:id/confirm	Yes	Admin, Sales
PATCH	/challans/:id/cancel	Yes	Admin, Sales

7. Roles & Permissions Matrix

Four roles define access across all modules. The backend enforces all permission checks at the API layer as the authoritative source.

Action	Admin	Sales	Whse	Accts
Customers:	Y	Y	-	-
Create				
Customers:	Y	Y	Y	Y
Read				
Customers:	Y	Y	-	-
Update				
Add Follow-	Y	Y	-	-
up				
Products:	Y	-	-	-
Create				
Products:	Y	Y	Y	Y
Read				
Products:	Y	-	Stock	-
Update				
Stock:	Y	-	Y	-
Adjust				
Image	Y	-	Y	-
Upload				
Challan	Y	Y	-	-
Create				
Challan	Y	Y	Y	Y
Read				
Challan	Y	Y	-	-
Confirm				
Challan	Y	Y	-	-
Cancel				
Download	Y	Y	-	-
PDF				

8. Setup Instructions

Prerequisites

- Node.js 20+
- PostgreSQL 16 (or Docker)
- npm

Local Setup (without Docker)

```
git clone https://github.com/raj-singh1802/erp-crm-operations-portal.git
cd erp-crm-operations-portal
```

```
# Backend setup
cd backend
cp .env.example .env
npm install
npx prisma migrate deploy
npx prisma db seed
npm run start:dev

# Frontend setup (separate terminal)
cd frontend
cp .env.example .env
npm install
npm run dev
```

Setup with Docker

```
docker compose up --build

# Run migrations & seed:
docker exec erp_crm_api npx prisma migrate deploy
docker exec erp_crm_api npx prisma db seed

# Frontend:
cd frontend && npm install && npm run dev
```

9. Deployment Guide

The application is deployed on free-tier infrastructure. Two options are available:

Option A: Local Setup

Follow Section 8 (Setup Instructions). The assignment accepts local setup + screen recording as an alternative to live deployment.

Option B: Deploy to Free Tier (Reference)

1. Database (Neon)

Create a Neon project. Copy the pooled connection string (DATABASE_URL) and direct connection string (DIRECT_URL). Run: `npx prisma migrate deploy` & `npx prisma db seed`.

2. Backend (Render)

Create a new Web Service, connect GitHub repo, root directory backend/. Build: `npm install` & `npx prisma generate` & `npm run build`. Start: `npx prisma migrate deploy` & `node dist/main`. Set env vars: DATABASE_URL, DIRECT_URL, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, CORS_ORIGIN.

3. Frontend (Vercel)

Import GitHub repo, root: frontend/. Framework preset: Vite. Env var: VITE_API_BASE_URL = Render backend URL. Deploy.

10. Test Credentials

All seeded users share the same password for testing purposes.

Role	Email	Password
Admin	admin@test.com	password123
Sales	sales@test.com	password123
Warehouse	warehouse@test.com	password123
Accounts	accounts@test.com	password123

11. Assumptions & Design Decisions

Auth Token Storage

Access and refresh tokens are stored in JavaScript variables (in-memory), not in localStorage. On page refresh, the user must log in again. This is deliberate - in-memory storage is immune to XSS-based token theft at the cost of session persistence.

Snapshot Timing

Product name and price are snapshotted into ChallanItem only at confirm time, not at draft creation. This avoids stale-price risk from long-lived drafts.

Cancellation Rule

Only Draft challans can be cancelled. Confirmed challans cannot be cancelled (no stock reversal). Avoids inventory correction complexity.

Rate Limiting

Not implemented. Would use @nestjs/throttler in production. Out of scope for the 48-hour build window.

Refresh Token Rotation

On each refresh, a new pair is issued and the old refresh token hash is replaced. No token family tracking.

12. Known Limitations

- No automated tests - No unit or e2e tests beyond manual smoke-testing.
- No rate limiting - API is unprotected against brute-force or high-frequency requests.
- No refresh token theft detection - Rotation but no family/session table.
- No pagination on stock movements - Returns all records unfiltered.
- No image upload validation - No file type, size, or filename sanitization.
- No createdBy on Customer records - Unlike Challan and StockMovement.
- Basic PDF styling - Uses Helvetica fonts only.
- Docker does not auto-run migrations - Requires manual docker exec.

13. Submission Checklist

- GitHub repository: <https://github.com/raj-singh1802/erp-crm-operations-portal>
- Live frontend URL: <https://erp-crm-operations-portal.vercel.app>
- Live backend URL: <https://erp-crm-api.onrender.com>
- Test credentials for all 4 roles
- Postman collection (postman/ERP_CRM_Portal.postman_collection.json)
- README with architecture, setup, and deployment instructions
- Known limitations documented
- Screen recording: <https://www.loom.com/share/11cea46c45d74478a4ef191f2cb07a3d>